

Project description

1 Project title

Agile Requirement and Change Management System for Japan–Vietnam Offshoring Projects

2 Customer description

Solashi Japan LLC is an information technology company specializing in providing software development solutions and digital services to Japanese enterprises through an offshore delivery model. Headquartered in Japan, the company focuses on delivering high-quality, cost-effective, and reliable IT solutions by collaborating closely with its development center in Vietnam.

The company's organizational structure reflects a typical Japan–Vietnam offshoring model. Strategic direction and client relationship management are led by the Chairman and Japanese Project Managers based in Japan, who work directly with Japanese customers. Operational execution is handled by the Vietnam branch, managed by a Vietnamese CEO and CTO, who oversee project delivery, technical direction, and resource allocation.

Solashi Japan LLC operates with cross-functional teams, including Vietnamese BrSEs, designers, quality assurance engineers, and large-scale technical teams covering web, mobile, database, and AI development. Supporting functions such as Human Resources and Accounting ensure stable operations. This structure enables effective communication, strong requirement control, and consistent delivery quality for Japanese clients.

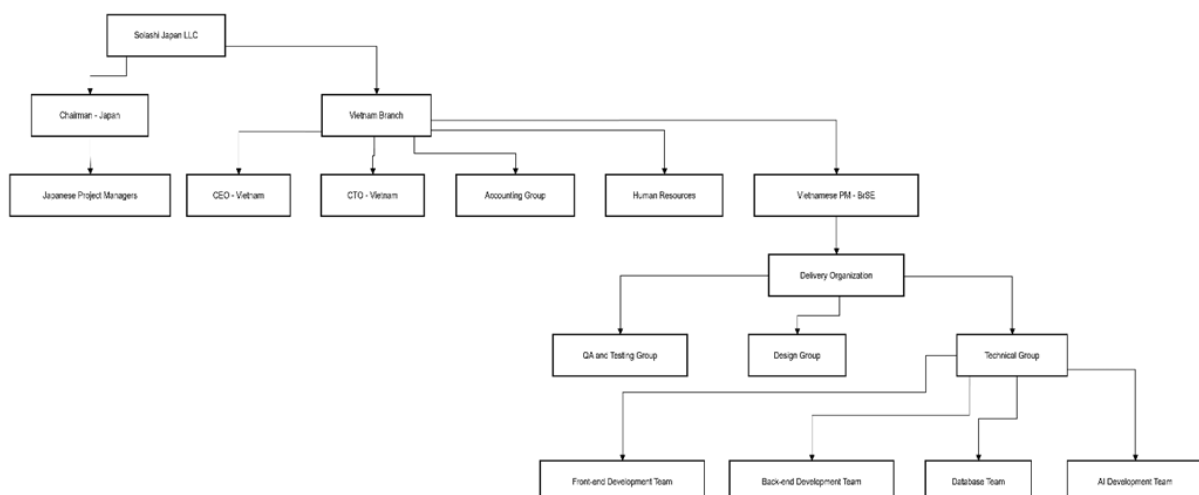


Figure 1: Solashi Japan LLC structure

3 Business Process

1. Company Functional Areas and Business Processes

Solashi Japan LLC operates through several functional areas that collaborate closely to deliver software projects for Japanese customers under an offshore model.

• Management and Project Governance:

This area includes the Chairman, Japanese Project Managers, Vietnamese CEO, CTO, and Vietnamese BrSE. It is responsible for strategic direction, project approval, resource allocation, progress monitoring, and risk management.

• Business Analysis and Communication (BrSE Group):

The BrSE group acts as a bridge between Japanese customers and Vietnamese technical teams. Their responsibilities include requirement elicitation, clarification, translation of business requirements into technical specifications, and ongoing communication throughout the project lifecycle.

• Design and Technical Development:

This area consists of the Design team and Technical teams, including Front-end, Back-end, Database, and AI Development teams. They are responsible for system design, software development, and implementation based on approved requirements.

• Quality Assurance (QA):

The QA group ensures product quality by preparing test cases, executing functional and regression testing, and validating deliverables against requirements.

• Support Functions:

Human Resources and Accounting support internal operations, staffing, and financial management, ensuring stable project execution.

2. Core Cross-Functional Business Process

• Software Development and Delivery Process

The core cross-functional business process of the company is the **Software Development and Delivery Process**, which involves coordination among management, BrSE, technical teams, and QA.

• Process Explanation

1. The process starts when a Japanese customer submits a project request to Solashi Japan LLC.

2. Japanese Project Managers evaluate the request and coordinate with the Chairman to approve the project.
3. The approved project is handed over to the Vietnam branch, where the CEO and CTO assign a Vietnamese BrSE and allocate resources.
4. The BrSE group collects and analyzes requirements, then confirms them with Japanese Project Managers.
5. The Design team prepares system and UI/UX designs based on clarified requirements.
6. Technical teams develop the system components in parallel.
7. The QA group conducts testing to verify quality and compliance with requirements.
8. If defects are found, the system is returned to the technical teams for correction.
9. Once approved, the final product is delivered to the Japanese customer.

Although Solashi's real-world organizational structure includes Project Managers, the prototype system developed in this project focuses specifically on the requirement and change management layer. Therefore, the system models only the operational roles directly involved in requirement clarification and technical implementation, such as BrSE, Developers, and QA. This process ensures transparency, quality control, and effective collaboration across functional areas.

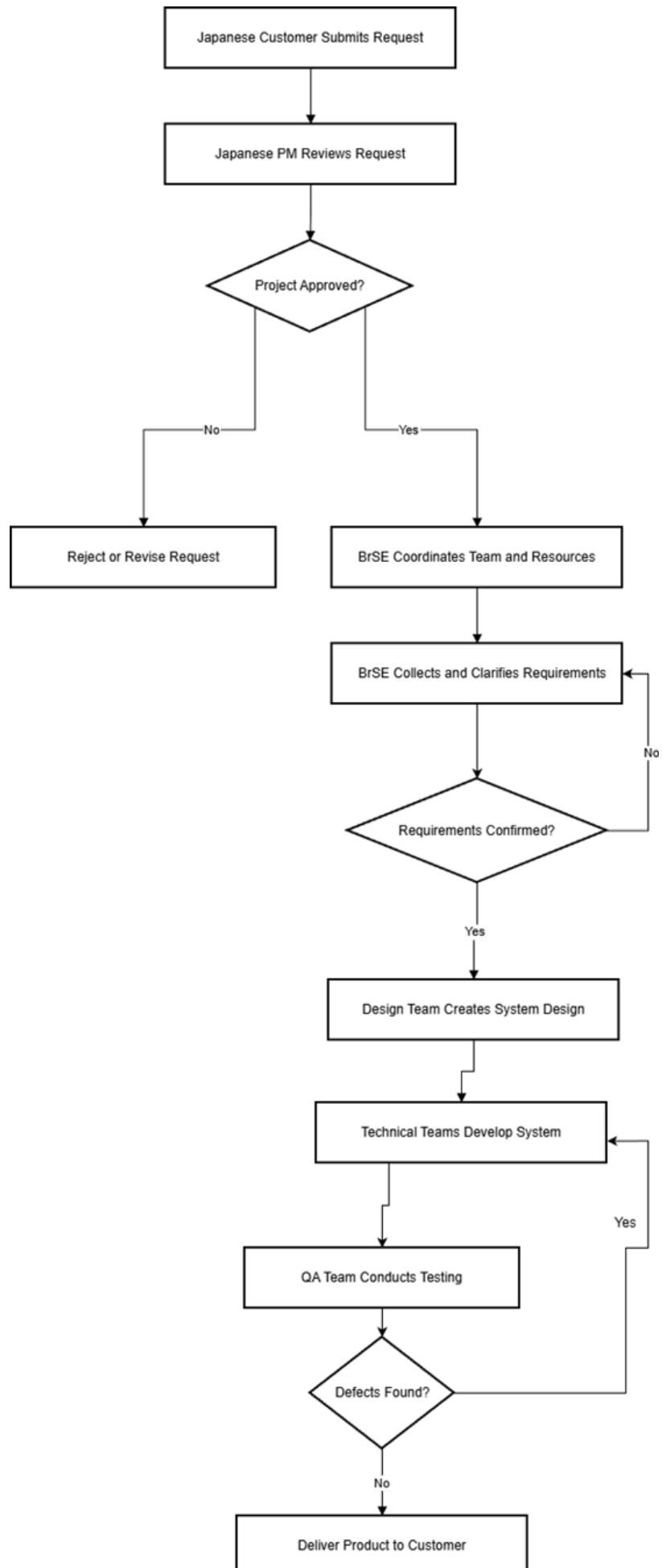


Figure 2: Solashi Japan LLC core business process

4 Project summary

This project aims to design and develop a prototype **Agile Requirement and Change Management System (Student Edition)** to support Sprint-based software development in a Japan–Vietnam offshoring environment. The system focuses on managing requirements and Change Requests (CRs) as structured governance processes embedded within Scrum cycles, rather than treating them as informal discussions conducted through emails or chat tools.

The proposed system introduces a centralized platform with strict role-based access control (RBAC) to regulate responsibilities among Customer representatives, BrSE, Execution team (Developers, QA), and System Administrators. In this prototype system, the traditional Project Manager role is intentionally excluded. Instead, the BrSE (Bridge System Engineer) acts as the coordination role between the customer and the development team, reflecting common practices in Japan–Vietnam offshore software projects. All incoming items are categorized into predefined issue types—Bugs, New Features, and Change Requests—and each type follows a clearly defined state-driven lifecycle. In particular, Change Requests must undergo a formal Impact Analysis process before any approval decision is made. This analysis includes effort estimation, risk assessment, and evaluation of schedule implications, ensuring that no CR is directly inserted into an active Sprint without structured review.

Unlike traditional ticketing tools that focus primarily on task tracking, this system emphasizes Sprint-level governance. Approved Change Requests are allocated either to the Product Backlog or to a specific Sprint through controlled decision gates aligned with Sprint Planning practices. Once a Sprint is active, change insertion is restricted and requires explicit re-evaluation to protect Sprint commitment stability. All state transitions are governed by predefined triggers and are recorded through audit logging to ensure traceability and accountability.

The significance of this project lies in addressing structural risks commonly observed in Japan–Vietnam offshore collaborations, including ambiguous change ownership, scope creep during active Sprints, and person-dependent communication through intermediaries such as BrSEs. By embedding Agile principles into a controlled state-machine workflow, the system demonstrates how flexibility can coexist with governance. As an academic prototype inspired by enterprise best practices, the solution does not attempt to replace existing execution tools (e.g., Jira) but instead provides a decision-management layer that enhances transparency, strengthens accountability, and promotes more predictable Sprint outcomes in offshore project environments.

5 Significance and innovation

1, Why did you undertake the project?

This project was undertaken to address a recurring structural problem in Japan–Vietnam offshore software development: the lack of a Sprint-aware and governance-driven mechanism for managing requirements and Change Requests (CRs) as formal business decisions.

In many offshore collaborations, requirements and changes are discussed informally through emails, chat platforms, or comments within task-tracking tools such as Jira. While these tools support execution tracking, they often do not enforce structured decision gates before a request affects Sprint commitments. A frequent issue is the misclassification of requests—particularly confusion between Bug fixes, New Features, and scope-changing Change Requests. When such distinctions are not formally defined at submission, ambiguity arises regarding ownership, estimation responsibility, and impact on the current Sprint.

For example, a customer may propose a modification during an active Sprint without a clear understanding of its schedule implications. If this request is treated as a minor adjustment rather than a scope-altering CR, it may disrupt Sprint stability, invalidate prior commitments, and create friction between customer representatives and BrSE, and Execution Team . These problems are amplified in offshore environments where communication gaps and time zone differences increase the risk of misunderstanding.

Therefore, this project aims to design a centralized Agile-based prototype that integrates structured CR governance directly into the Scrum lifecycle. By enforcing explicit issue categorization, mandatory impact analysis, and controlled Sprint allocation rules, the system demonstrates how change flexibility can coexist with Sprint discipline in offshore collaborations.

2, What is innovative about the system?

The innovation of the proposed system lies not in complex infrastructure or advanced automation, but in its decision-centric and Sprint-aware workflow architecture. Unlike conventional academic management systems that primarily focus on ticket tracking, this project emphasizes governance at the point where requirements influence Sprint commitments.

A key innovative aspect is the integration of a state-driven lifecycle model for each Issue Type. Every request—whether Bug, New Feature, or Change Request—follows a predefined state machine with controlled transitions triggered by authorized roles. In particular, Change Requests must pass through a structured Impact Analysis stage before they can be approved or allocated to a Sprint. This ensures that effort estimation, risk assessment, and timeline impact are formally evaluated prior to affecting development plans.

Another innovative element is the protection of Sprint stability through controlled insertion rules. Once a Sprint is active, new Change Requests cannot be directly injected into the Sprint Backlog without explicit re-evaluation and approval. This design embeds Scrum discipline into the governance layer, preventing uncontrolled scope expansion while maintaining adaptability.

The clarification loop (Submitted ↔ Pending Info) further enhances innovation by institutionalizing requirement clarification as an auditable state rather than an informal discussion. Combined with role-based access control (RBAC) and full audit logging of state transitions, the system ensures traceability, accountability, and clearly defined decision ownership across Customer and BrSE, and Execution Team roles.

Collectively, these design principles demonstrate how structured decision governance can be embedded into Agile environments without introducing enterprise-level complexity.

3, What new methodologies or technologies are developed?

From a methodological perspective, the project applies Scrum principles—such as Product Backlog management, Sprint Planning alignment, and Sprint commitment protection—while embedding a formal Change Request governance layer above the execution level. The system models how CR lifecycle management can operate in parallel with iterative development cycles without compromising agility.

The methodological contribution lies in combining three core concepts within a single prototype:

- A Sprint-aware Change Request lifecycle governed by a state machine
- Role-based decision gates aligned with Scrum responsibilities
- Structured impact analysis before backlog or Sprint allocation

Rather than introducing new technologies, the project integrates standard web development practices into a cohesive prototype that simulates real-world governance scenarios. The system includes:

- A role-based access control mechanism that regulates actions at project and Sprint levels
- A workflow engine implementing controlled state transitions and trigger conditions
- An Impact Analysis module capturing estimation, risk level, and schedule implications
- An audit logging mechanism ensuring traceable decision history
- A relational database schema modeling Company → Project → Sprint → Issue hierarchy

The technical implementation focuses on designing a RESTful backend capable of enforcing lifecycle constraints and permission logic, while the frontend dynamically adapts available actions based on user role and current issue state.

Overall, the contribution of this project lies in demonstrating how Agile flexibility and structured governance can be systematically integrated within a Sprint-based offshore collaboration model. As a Student Edition prototype inspired by enterprise practice, the system does not attempt to replace full-scale project management platforms, but instead illustrates a practical governance layer tailored to the specific risks of Japan–Vietnam offshoring environments.

6 Approach

1, Agile Methodology and Sprint Structure

The project adopts the Scrum framework as its development methodology and organizes implementation into three academic sprints. This approach reflects iterative development principles while simultaneously modeling how Sprint-based governance can be embedded into requirement and Change Request (CR) management within offshore collaborations.

Beyond using Scrum as a development practice, the system itself is designed to be Sprint-aware. The prototype models a hierarchical structure consisting of Company → Project → Product Backlog → Sprint → Issue. Change Requests are not treated as standalone tickets but as governance-controlled items that may influence Sprint commitments only after structured evaluation.

Each sprint of development delivers a progressively enhanced version of the governance layer, allowing validation of the CR lifecycle, role-based decision gates, and Sprint allocation rules. This iterative structure ensures that both technical implementation and workflow logic evolve through feedback and refinement, consistent with Agile principles.

2, Sprint Objectives and Deliverables

Sprint 1 – Governance Foundation and Core Lifecycle

The primary objective of Sprint 1 is to establish the structural and logical foundation of the system. Key deliverables include:

- Implementation of user authentication and role-based access control (RBAC) across Customer, BrSE, Execution team (Developer, QA), and System Admin roles.
- Design of the relational database schema modeling Company, Project, Sprint, Product Backlog, Issue, and Status entities.
- Development of the Issue classification mechanism requiring explicit selection among Bug, New Feature, and Change Request at submission.
- Implementation of a state-driven lifecycle engine with predefined status transitions.
- Establishment of the clarification loop (Submitted ↔ Pending Info) as a controlled state within the workflow.

By the end of Sprint 1, the system provides a functional prototype capable of structured issue submission and controlled state transitions governed by role permissions.

Sprint 2 – Impact Analysis and Sprint Allocation Governance

Sprint 2 focuses on embedding formal decision-making and Sprint-level protection mechanisms into the workflow. Key deliverables include:

- Development of the Impact Analysis module for Change Requests, capturing effort estimation (man-days), risk level, affected components, and projected schedule impact.
- Implementation of conditional workflow constraints ensuring that Change Requests must complete Impact Analysis before entering approval stages.
- Introduction of controlled approval gates where authorized roles (BrSE and Customer Representative) can approve, reject, or defer requests.
- Implementation of Sprint allocation rules, enabling approved CRs to be assigned either to the Product Backlog or to a specific Sprint.
- Enforcement of Sprint stability constraints, preventing direct insertion of new CRs into an active Sprint without explicit re-evaluation.

At the end of Sprint 2, the system supports governance-driven and traceable Change Request handling aligned with Scrum Sprint commitments.

Sprint 3 – Traceability, Reporting, and System Consolidation

Sprint 3 focuses on strengthening transparency, usability, and prototype stability. Key deliverables include:

- Implementation of audit logging to record state transitions, decision actors, timestamps, and approval history.
- Development of filtering and reporting features to provide visibility across Projects and Sprints.
- Refinement of role-based UI behavior, ensuring available actions dynamically reflect both user role and current issue state.
- System testing, bug fixing, and performance stabilization.
- User Acceptance Testing (UAT) and preparation of the final academic demonstration.

By the end of Sprint 3, the project delivers a consolidated prototype demonstrating structured, Sprint-aware requirement and Change Request governance tailored to offshore environments.

3, Release Strategy

Each development sprint concludes with a Sprint Review, during which the working increment of the system is demonstrated for evaluation and feedback. Observations from each review inform backlog refinement and prioritization for subsequent sprints.

The final release integrates all core governance mechanisms—including issue classification, state-machine lifecycle management, impact analysis, role-based approval gates, Sprint allocation control, and audit logging—into a coherent academic prototype. The system demonstrates how structured governance can coexist with Agile flexibility without replacing existing task execution platforms.

4, Technologies and Tools

The prototype is developed using practical and student-feasible web technologies while focusing primarily on workflow logic and governance modeling rather than enterprise-scale infrastructure.

The system architecture consists of:

- A RESTful backend responsible for enforcing lifecycle rules, role-based access control, state transition constraints, and Sprint allocation logic.
- A relational database schema modeling Company → Project → Sprint → Issue relationships and maintaining audit history.
- A frontend interface that dynamically adapts available actions based on user role and issue state, reflecting governance constraints visually.

Standard version control and collaboration tools are used to manage source code and sprint planning within the development team.

The selected technical approach ensures feasibility within the academic timeframe while enabling the demonstration of structured decision governance, Sprint-level protection mechanisms, and traceable Change Request management within an Agile offshore collaboration model.

7 Expected outcomes

The proposed Agile Requirement and Change Management System (Student Edition) is expected to deliver a functional academic prototype that demonstrates how Sprint-aware governance can improve requirement clarity and Change Request (CR) control in a Japan–Vietnam offshore development context. The primary outcome is the establishment of a structured, role-governed, and traceable lifecycle in which requirements and changes are treated as formal decision states aligned with Scrum practices, rather than informal technical discussions.

From an operational perspective, the system is expected to reduce ambiguity in request handling by enforcing explicit issue classification into Bugs, New Features, and Change Requests at submission. Each issue follows a predefined state-machine lifecycle with controlled transitions triggered by authorized roles. In particular, Change Requests must complete a structured Impact Analysis stage—covering effort estimation, risk assessment, and schedule implications—before entering approval gates or affecting Sprint commitments. This governance mechanism ensures that no CR is introduced into a Sprint without formal evaluation, thereby protecting Sprint stability and reducing uncontrolled scope expansion.

The system is also expected to demonstrate how Change Requests can be systematically allocated either to the Product Backlog or to a specific Sprint through controlled decision gates. Once a Sprint is active, insertion of new scope requires explicit re-evaluation, reinforcing Scrum discipline while maintaining flexibility through structured review. By embedding these rules within the workflow engine, the prototype illustrates how Agile adaptability can coexist with governance constraints.

Through role-based access control (RBAC), the system ensures that only authorized roles—such as BrSE and Customer Representatives—can conduct impact analysis, approve changes, or alter Sprint allocations. Combined with audit logging of all state transitions, decision actors, and timestamps, the prototype enhances accountability, traceability, and decision ownership across both Customer and stakeholders.

From an academic and organizational learning perspective, the expected outcome is a demonstrable model that illustrates how offshore project teams can balance iterative development with structured change governance. By centralizing issue records, decision history, and Sprint allocation logic within a unified platform, the system reduces reliance on fragmented communication channels and clarifies responsibility boundaries in cross-border collaborations.

Overall, the expected outcome is not a full-scale enterprise solution but a stable, coherent, and demonstrable prototype that reflects real-world governance challenges in Sprint-based offshore projects. The project showcases how structured state-driven workflows, formal impact evaluation, and role-controlled approval mechanisms can contribute to clearer decision-making and more predictable Sprint execution within an academically feasible scope.

8 Description of personnel

PART 1: STAKEHOLDERS

In this project, the key stakeholders include **Solashi Company (representing the industry client)** and **the University (representing academic quality and educational standards)**. Their involvement ensures that the project is both practically relevant to real-world business environments and compliant with academic requirements.

Stakeholder	Role	Responsibilities & Contributions
Mr. Do Tien Thanh (CTO – Solashi)	Client / Product Owner	- Acts as the owner of business requirements and product vision. - Provides real-world business requirements related to the Change Request management process. - Approves or rejects system features during Sprint Reviews. - Participates in User Acceptance Testing (UAT) to validate whether the system meets operational standards.

PART 2: PROJECT TEAM (DETAILED DESCRIPTION OF FIVE MEMBERS)

The project team consists of five members operating under the Agile framework. Each role is clearly defined and directly linked to specific functions of the **Agile Requirement and Change Request Management System**.

1. Project Manager (PM) – Do Thanh Long

Role: Scrum Master & Project Lead

Responsibilities:

- Plans and facilitates Sprint Planning, Daily Scrum, Sprint Review, and Sprint Retrospective meetings.
- Acts as the single Point of Contact between the development team and the Solashi client.
- Manages project risks, resolves internal conflicts, and ensures adherence to project timelines.

Contribution:

- Takes full accountability for successful project delivery and completion of all agreed deliverables.

2. Business Analyst (BA) – Phung Quoc Viet

Role: Business Liaison

Responsibilities:

- Conducts requirement elicitation sessions with the client to gather business needs.
- Prepares Software Requirements Specification (SRS), Use Cases, User Stories, and maintains the Product Backlog.
- Designs UI mockups and wireframes using Figma to align business expectations before development begins.

Contribution:

- Ensures that the development team correctly understands business processes, minimizing the risk of requirement misinterpretation.

3. Backend Developer – Man Ngoc Dac

Role: Core System & Database Engineer

Responsibilities:

- Designs the database schema to store Change Requests, user data, workflow states, and audit logs.
- Develops RESTful APIs to handle business logic such as approval workflows, access control, and effort estimation.
- Supports system integrations (e.g., email notifications) when applicable.

Contribution:

- Ensures system stability, data integrity, security, and accurate execution of core business logic.

4. Frontend Developer – Nguyen Viet Nhat

Role: User Interface Engineer

Responsibilities:

- Implements UI designs from Figma using modern frontend frameworks such as ReactJS.
- Develops interactive features including Change Request forms and Kanban-style workflow boards.
- Optimizes usability and user experience to meet expectations of Japanese enterprise users.

Contribution:

- Delivers a professional, intuitive, and user-friendly interface that enhances overall system adoption.

5. Tester / Quality Control (QC) – Le Huu Dang

Role: Quality Assurance Specialist

Responsibilities:

- Develops Test Plans and Test Cases based on BA documentation and system requirements.
- Conducts functional testing, UI testing, and regression testing across all system modules.
- Logs defects, tracks resolution progress, and verifies fixes before release.

Contribution:

- Ensures the delivered Change Request Management System meets quality standards and is as defect-free as possible, protecting the team's credibility.

PART 3: RACI MATRIX

To demonstrate professional project governance, a RACI Matrix is applied as follows:

Task / Activity	PM	BA	Dev	Tester	Client (Solashi)
Requirements Elicitation	A	R	I	I	C
Database Design	I	C	R	I	I
Feature Development	I	I	R	I	I
Testing	I	C	I	R	I
User Acceptance Testing (UAT)	A	C	I	I	R

R – Responsible; **A** – Accountable; **C** – Consulted; **I** – Informed

9 Feasibility

The proposed Agile-based Requirement and Change Management System is considered feasible from technical, operational, and schedule perspectives.

Technical Feasibility

The system is technically feasible as it is based on well-established web technologies and Agile development practices. The project scope focuses on core functionalities such as requirement management, change request handling, role-based access, and traceability, which can be implemented using commonly used frameworks and programming languages. The development team possesses the necessary technical skills to design, develop, and test the system within the defined scope.

Operational Feasibility

From an operational perspective, the system aligns well with the working practices of IT offshoring companies serving Japanese customers. The Agile methodology supports iterative delivery and continuous feedback, while the structured change request process ensures controlled adaptation to requirement changes. The system can be easily integrated into existing project workflows without significant disruption to daily operations.

Schedule and Resource Feasibility

The project is feasible within the planned three-sprint schedule, as tasks are clearly divided among team members with defined roles, including Project Manager, Business Analyst, Developers, and Tester. The team size and skill distribution are appropriate for delivering a functional prototype. Overall, the project is achievable within the available time, resources, and academic constraints.